

Replication: Deep Learning-Based Causal Inference for Large-Scale Combinatorial Experiments

Ye, Zhang, Zhang, Zhang, and Zhang (Management Science, 2025), with causalmetrics and PyTorch

Weiheng Zhang

September 05, 2026

Contents

1	The paper in one page	2
2	Data and ground truth	2
2.1	The synthetic data	2
2.2	Figure 1: heterogeneous response to two experiments	3
2.3	Table 2: ground-truth ATEs	3
3	The structured model and its influence function	4
3.1	Link function and gradients	4
3.2	Why the correction removes first-order bias	4
3.3	The package function	4
4	First stage: the structured network	5
5	Second stage: DeDL and SDL	5
5.1	The ridge on $\Lambda(x)$ is not optional	5
5.2	Table 7: fold-by-fold estimates	6
5.3	Figure 5: DeDL against SDL	8
6	Benchmarks and metrics	8
6.1	LA, LR, and PDL	8
6.2	Table 3: comparison of the estimators of the ATE	9
6.3	Table 4: best-treatment identification	9
7	Conditional effects by demographic group	10
8	Implementation lessons	12
8.1	Figure 7: the training error as the compass	12
8.2	The authors' notebook against the paper's algorithm	13
9	Appendix D: synthetic experiments	14
10	What matched, what did not, and what we learned	16
11	Appendix: formulas used	16
11.1	Gradients of the generalized sigmoid	16
11.2	The authors' inference conventions	16

1 The paper in one page

The question is how to estimate the effect of a combination of treatments that was never tested. Large platforms run hundreds of A/B tests at once under orthogonal assignment, so every user sits in some combination of treatments, but only a few combinations have enough traffic to be measured on their own. The common practice adds up the single-treatment effects. Ye, Zhang, Zhang, Zhang, and Zhang (2025) show on a video platform that this linear addition can be badly wrong (their Figure 1), and propose a method, debiased deep learning (DeDL), that observes only $m + 2$ combinations of m treatments and recovers the average treatment effect (ATE) of all 2^m .

The method has three parts.

1. **A structured outcome model.** $E[Y \mid X = x, T = t] = G(\theta(x), t)$ with a known link G and an unknown parameter function $\theta(x)$. Their choice is the generalized sigmoid, $G(\theta, t) = \theta_{m+1} / (1 + \exp(-(\theta_0 + \theta_1 t_1 + \dots + \theta_m t_m)))$, which captures diminishing or increasing returns to combining treatments user by user (their Theorem 1 shows it can approximate any ATE function as the number of distinct covariate profiles grows). Two ReLU networks output $\theta_0(x)$ and $(\theta_1, \dots, \theta_m)(x)$; the scale θ_{m+1} is one trained constant.
2. **An influence function.** The plug-in estimator $\frac{1}{n} \sum_i [G(\hat{\theta}(x_i), t) - G(\hat{\theta}(x_i), t_0)]$ inherits the bias of the network. Following Farrell, Liang, and Misra (2020) they add the Neyman-orthogonal correction (their Proposition 2),

$$\psi_i(t) = H_i - H'_{\theta,i} \Lambda(x_i)^{-1} \ell_{\theta,i}, \quad H_i = G(\hat{\theta}_i, t) - G(\hat{\theta}_i, t_0),$$

where H_θ is the gradient of H in θ , $\Lambda(x) = 2 E_\nu[G_\theta G'_\theta \mid x]$ averages the outer product of the gradient over the *known* assignment distribution ν of the observed combinations, and $\ell_\theta = 2G_\theta(\hat{\theta}, \check{t})(G(\hat{\theta}, \check{t}) - y)$ is the gradient of the squared loss at the realized treatment $\check{t}$. The mean of $\psi_i(t)$ over a fold whose $\hat{\theta}$ was trained on the other folds is root- n normal (their Theorem 2).

3. **Four-fold cross-fitting**, the best combination as the largest estimated ATE, and the gap $\tau(t) = \mu(t^*) - \mu(t)$ with its own influence function for best-treatment identification.

Benchmarks: linear addition (LA), linear regression of y on (x, t) (LR), a pure network on (x, t) (PDL), and the plug-in structured network without the correction (SDL). On the platform’s data the “unobserved” combinations are in fact observed, so every estimator is scored against the ground truth with four metrics: the correct-direction ratio (sign and significance), MAPE, MSE, and MAE. DeDL wins on every metric (their Table 3), identifies the best combination (Table 4), and does so within demographic groups (Tables 5-6). Their Section 5 lists the implementation checkpoints: stratified sampling to balance covariates, an architecture and learning-rate search, a ridge on $\Lambda(x)$ before inversion, and the training error as the compass (Figure 7).

What this note does. The platform data are proprietary. The authors ship a synthetic data set with the same structure, a 999-subgroup file for Figure 1, and a Keras notebook whose printed outputs are the numbers we match. We reproduce every table and figure of the empirical section in the published layout, using `est_dml_structured()` from `causalmetrics` for the influence function and inference and a PyTorch script for the network, then port the validation experiment of the paper’s Appendix D. Two things are learned on the way: the correction term is unstable without a ridge (Section 5.1), and the authors’ notebook departs from the paper’s algorithm in ways that change the ranking of DeDL and SDL (Section 8.2).

2 Data and ground truth

2.1 The synthetic data

The file has 50,000 users, the three treatment indicators, an outcome y scaled to $[0, 1]$, 87 covariates (10 standardized continuous usage variables and 77 dummies for operating system, activity, gender, age, location,

follower and friend counts, and phone price tier), and a fold index. Unlike the paper’s stratified sample, all eight combinations are present in every fold.

Table 1: Users per treatment combination and fold in the synthetic data. The exercise treats the last three combinations as unobserved.

Treatment combination	Status in the exercise	Fold 1	Fold 2	Fold 3	Fold 4
(0, 0, 0)	Observable	1613	1561	1590	1579
(0, 0, 1)	Observable	1576	1549	1578	1559
(0, 1, 0)	Observable	1505	1624	1555	1498
(1, 0, 0)	Observable	1527	1573	1549	1616
(1, 1, 1)	Observable	1579	1562	1620	1557
(1, 1, 0)	Unobservable	1470	1631	1531	1538
(1, 0, 1)	Unobservable	1552	1594	1512	1581
(0, 1, 1)	Unobservable	1553	1610	1521	1537

2.2 Figure 1: heterogeneous response to two experiments

The paper’s opening figure uses 1,254 subgroups of users defined by demographics and activity, each observed under all four combinations of two experiments (D and S). The released file has 999 subgroups with the mean outcome under (D,S), ($_ ,S$), (D, $_$), and ($_ ,_$). The left panel plots the true combined effect against the linear-additive prediction; the right panel the distribution of the percentage error of linear addition.

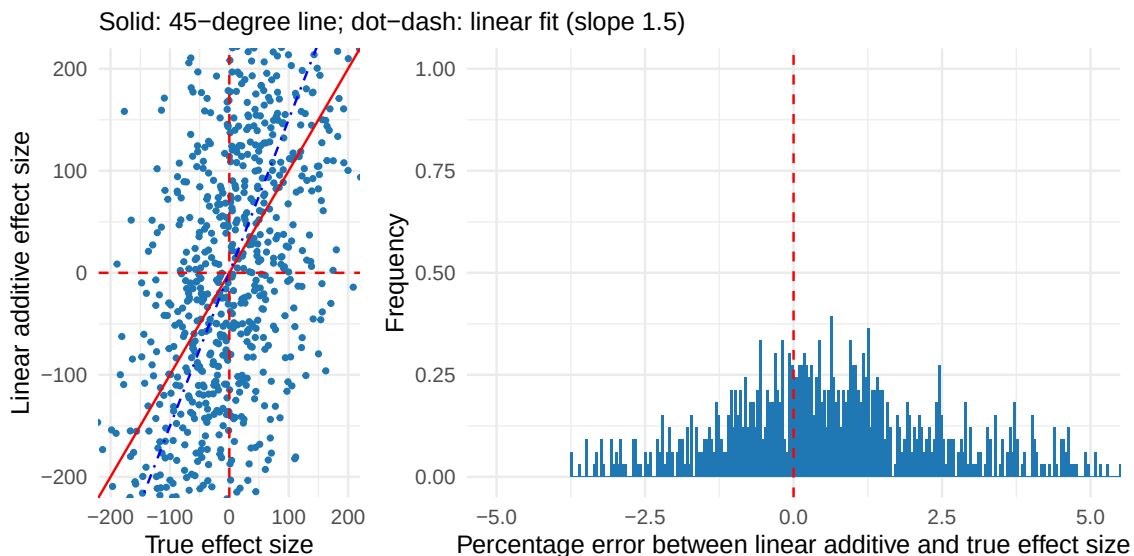


Figure 1: Heterogeneous response to two experiments (paper’s Figure 1). Left: true combined effect against the linear-additive effect by subgroup, with the 45-degree line (solid) and the no-intercept linear fit (dot-dash). Right: density of the percentage error of linear addition, trimmed to its 10 to 90 percent quantiles.

The fitted slope is 1.50 rather than one, and linear addition gets the sign of the combined effect wrong in 23 percent of the subgroups. This is the motivation for a model of interactions.

2.3 Table 2: ground-truth ATEs

With all eight cells observed, the ground truth is the difference in cell means against the baseline (0,0,0). The authors test each difference with a pooled t -test when Levene’s test does not reject equal variances and Welch’s test otherwise; we do the same.

```
truth <- ground_truth_table(dat$y, key)
```

Table 2: Ground-truth ATE and best treatment identification of eight treatment combinations (paper’s Table 2, synthetic data). Relative effect size is the cell-mean difference against the baseline divided by the baseline mean. * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$; **** $p < 0.0001$.

Treatment combination t'	Relative effect size, % (1)	Observed or not (2)	Number of users (3)
(0, 0, 0)	0.000	Observable	6,343
(0, 0, 1)	51.882****	Observable	6,262
(0, 1, 0)	-35.301****	Observable	6,182
(1, 0, 0)	-35.779****	Observable	6,265
(1, 1, 1)	14.928****	Observable	6,318
(1, 1, 0)	-58.522****	Unobservable	6,170
(1, 0, 1)	33.203****	Unobservable	6,239
(0, 1, 1)	37.990****	Unobservable	6,221

Note. The optimal treatment combination is (0, 0, 1).

The relative effects match the notebook’s printed values (0.519, -0.353 , -0.358 , 0.149, -0.585 , 0.332 for the first six non-baseline combinations). Every effect is significant at any conventional level, so the correct-direction ratio will only reward sign agreement, and the best combination is (0, 0, 1). The synthetic outcome is far less noisy than the platform’s screen time: the paper’s ground truth has two insignificant combinations and relative effects of one to two percent.

3 The structured model and its influence function

3.1 Link function and gradients

Write $u = \theta_0 + \theta' t$ and $s(u) = 1/(1 + e^{-u})$. The generalized sigmoid is $G(\theta, t) = c s(u)$ with $c = \theta_{m+1}$, so

$$G_\theta(\theta, t) = (c s(u)(1 - s(u)), c s(u)(1 - s(u)) t', s(u))',$$

a vector of length $m + 2$. The plug-in contrast is $H = G(\theta, t) - G(\theta, t_0)$ and $H_\theta = G_\theta(\theta, t) - G_\theta(\theta, t_0)$. With the assignment distribution ν uniform over the five observed combinations, $\Lambda(x) = \frac{2}{5} \sum_{t \in \mathcal{T}^o} G_\theta(\theta(x), t) G_\theta(\theta(x), t)'$ is a 5×5 matrix per user, a sum of five rank-one terms. It is invertible in principle but nearly singular for users whose sigmoid is saturated ($s(u)$ near 0 or 1), because then all but the last entry of G_θ are close to zero for every t . This is where the ridge of Section 5.1 enters.

3.2 Why the correction removes first-order bias

The plug-in $\frac{1}{n} \sum_i H_i$ has bias $E[H'_\theta(\hat{\theta} - \theta_0)]$ to first order. The score ℓ_θ has zero conditional mean at θ_0 and derivative $E[\ell_{\theta\theta} | x] = \Lambda(x)$, so $E[\Lambda(x)^{-1} \ell_\theta(\hat{\theta}) | x] \approx \hat{\theta}(x) - \theta_0(x)$: the correction term estimates the nuisance error from the residuals and subtracts its first-order effect. This is the same Neyman orthogonality as in lecture 04, written for a parameter *function* rather than a scalar nuisance. Cross-fitting supplies the independence between $\hat{\theta}$ and the scored observations that the argument needs.

3.3 The package function

`est_dml_structured()` takes the out-of-fold $\hat{\theta}$ matrix from any engine, the realized treatments, the target combinations, the link, the assignment distribution, and the fold index, and returns the influence functions, the fold-by-fold and pooled estimates, the plug-in estimates (SDL) from the same call, and the contrasts to the best combination. With the true θ the correction has mean zero and the estimator equals the plug-in contrast, which is the package’s unit test. The first stage is `inst/python/dedl_nuisance.py`, a PyTorch script mirroring the authors’ Keras architecture (two 3-layer, 20-node ReLU branches, a fixed sigmoid, a final linear layer without bias, Adam with learning rate 10^{-3} , batch 128, 50 epochs).

4 First stage: the structured network

The paper trains on the five observed combinations of three folds and scores the fourth. The script does this for every fold in one call and writes $\hat{\theta}$ for every user out of fold; runs are cached in `derived/`.

```
theta_paper <- py_run(
  output = file.path(derived, "theta_paper.csv"),
  history = file.path(derived, "history_paper.csv"),
  args = c("--model structured --y y --t is_DP,is_LP,is_FYP --x-all-others --fold kfold",
    "--train-cells 000,001,010,100,111 --arch two_branch --hidden 20 --depth 3",
    "--epochs 50 --batch 128 --lr 0.001 --val-split 0.1 --seed 1")
)
stopifnot(all(theta_paper$.row == dat$.row))
history_paper <- read.csv(file.path(derived, "history_paper.csv"))
theta_cols <- c("theta_0", "theta_1", "theta_2", "theta_3", "theta_c")
```

Table 3: First stage per fold: squared loss after 50 epochs on the training rows (five observed combinations of the other three folds) and on the 10 percent validation split, and the trained scale $\hat{c}$.

Held-out fold	Epochs	Training MSE	Validation MSE	Best validation MSE	Users scored	$\hat{c}$
1	50	0.00351	0.00610	0.00610	12,375	1.000
2	50	0.00276	0.00443	0.00443	12,704	1.000
3	50	0.00329	0.00489	0.00489	12,456	0.999
4	50	0.00292	0.00493	0.00493	12,465	1.002

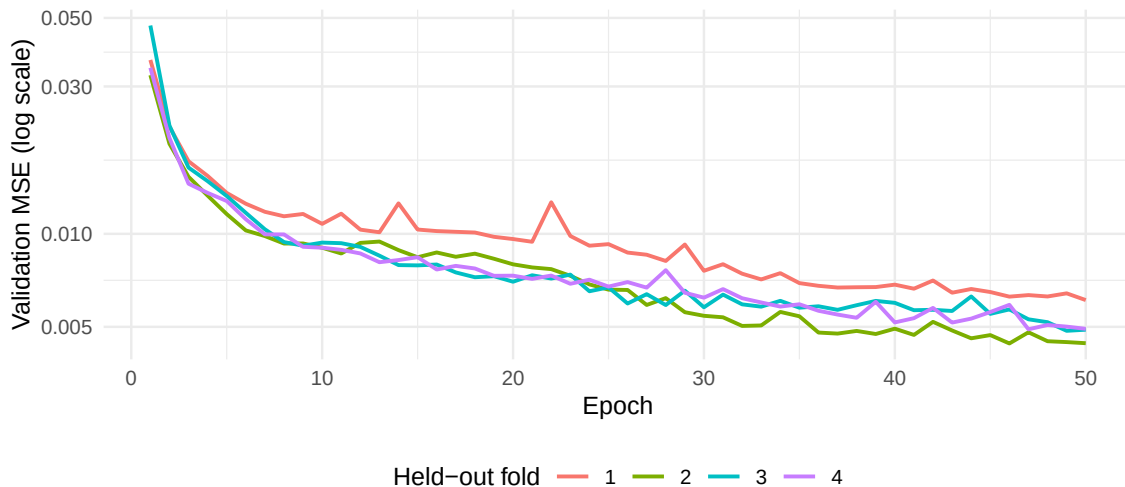


Figure 2: Validation mean squared error by epoch for the four cross-fitting networks.

The validation loss is still falling slowly at epoch 50 in every fold. The authors' Figure 7 and Section 5.3 make the point that the quality of this fit is the quality of everything that follows; Section 8.1 reproduces that figure.

5 Second stage: DeDL and SDL

5.1 The ridge on $\Lambda(x)$ is not optional

We first run the influence function exactly as written, with no ridge, and then with the ridge the paper mentions in Section 5.2 (they report grid searching values such as 0.001, 0.01, and 1; the notebook uses 0.5).

```

theta_mat <- as.matrix(theta_paper[, theta_cols])
ridge_grid <- c(0, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1)
ridge_res <- lapply(ridge_grid, function(r) {
  fit <- est_dml_structured(theta_mat, dat$y, dat[, tvar], targets = paper_order[-1], t_dist = t_dist_paper,
    fold_id = dat$fold, ridge = r, contrast_best = FALSE)
  p <- fit$estimates[fit$estimates$fold == "pooled", ]
  ape <- 100 * abs(p$estimate - truth$diff[-1]) / abs(truth$diff[-1])
  ape_sdl <- 100 * abs(p$plugin - truth$diff[-1]) / abs(truth$diff[-1])
  corr <- fit$psi - fit$plugin
  data.frame(ridge = r, mape_all = mean(ape), mape_unobs = mean(ape[5:7]), sdl_all = mean(ape_sdl), sdl_unobs = mean(ape_sdl[5:7]),
    max_corr = max(abs(corr)), share_large = mean(abs(corr) > 1), ill = fit$diagnostics$share_ill_conditioned)
})
ridge_tab <- bind_rows(ridge_res)

```

Table 4: Sensitivity of the DeDL estimates to the ridge added to $\Lambda(x)$ before inversion. The SDL columns do not depend on the ridge.

Ridge	DeDL				SDL	
	MAPE all, %	MAPE unobserved, %	Largest correction	Share correction > 1, %	SDL MAPE all, %	SDL MAP
0.000	16,077.76	37,494.42	5.29e+06	6.70	2.88	1.86
0.001	5.04	3.92	5.31e+01	1.98	2.88	1.86
0.010	3.11	1.46	6.70e+00	0.45	2.88	1.86
0.050	2.96	1.55	1.96e+00	0.02	2.88	1.86
0.100	3.00	1.70	1.32e+00	0.00	2.88	1.86
0.500	3.00	1.84	8.55e-01	0.00	2.88	1.86
1.000	2.98	1.85	6.11e-01	0.00	2.88	1.86

Without a ridge the estimator is useless: a handful of users with a saturated sigmoid have a nearly singular $\Lambda(x)$, their corrections run into the thousands, and the unobserved combinations' MAPE is in the tens of thousands of percent, while the plug-in is within two percent. A ridge of 0.01 removes those users' leverage and leaves the correction elsewhere intact; larger ridges shrink the correction toward zero, and DeDL converges to SDL. The rest of the note uses `ridge = 0.01`. This is the practical content of the paper's remark that " $\lambda(x)$ may be noninvertible for certain values of x ".

5.2 Table 7: fold-by-fold estimates

```

fit_dedl <- est_dml_structured(theta_mat, dat$y, dat[, tvar], targets = paper_order[-1], t_dist = t_dist_paper,
  fold_id = dat$fold, ridge = ridge_main)
fit_dedl
#> Structured double machine learning (link: gen_sigmoid, 3 treatments, 4 fold(s), n = 50000)
#> reference: (0, 0, 0); assignment distribution over 5 observed combinations; ridge = 0.01
#> target estimate plugin std.error conf.low conf.high
#> (0, 0, 1) 0.27825 0.27773 0.001559 0.27520 0.28131
#> (0, 1, 0) -0.18332 -0.18429 0.001181 -0.18564 -0.18101
#> (1, 0, 0) -0.19232 -0.19533 0.001096 -0.19446 -0.19017
#> (1, 1, 1) 0.08988 0.08693 0.001960 0.08604 0.09373
#> (1, 1, 0) -0.30812 -0.31302 0.001546 -0.31115 -0.30509
#> (1, 0, 1) 0.18222 0.18273 0.001950 0.17840 0.18604
#> (0, 1, 1) 0.20252 0.19887 0.001905 0.19879 0.20625
#> best target: (0, 0, 1)
# the plug-in (SDL) fold table from the same object
plugin_table <- function(fit) {
  crit <- qnorm(0.975)
  folds <- sort(unique(fit$fold_id))
  bind_rows(lapply(seq_along(fit$labels), function(k) {
    fe <- sapply(folds, function(f) mean(fit$plugin[fit$fold_id == f, k]))
    fs <- sapply(folds, function(f) sd(fit$plugin[fit$fold_id == f, k]) / sqrt(sum(fit$fold_id == f)))
    bind_rows(data.frame(target = fit$labels[k], fold = as.character(folds), estimate = fe, std.error = fs,
      conf.low = fe - crit * fs, conf.high = fe + crit * fs),
      data.frame(target = fit$labels[k], fold = "pooled", estimate = mean(fe), std.error = sqrt(sum(fs^2)) / length(folds)
    )
  })
}

```

```

    conf.low = mean(fe) - crit * sqrt(sum(fs^2)) / length(folds), conf.high = mean(fe) + crit * sqrt(sum(fs
  )))
}
sdl_tab <- plugin_table(fit_dedl)

```

Table 5: Detailed results of fourfold DeDL estimators (paper’s Table 7 layout). Effects are in units of the scaled synthetic outcome; the Mean row averages the fold estimates and the fold confidence bounds, as in the paper. SE and AE are scaled by constants, as in the paper.

Treatment combination	Ground-truth ATE (1)	Fold (2)	Estimated ATE (3)	95% CI for ATE estimate (4)	APE, % (5)	SE $\times 10^4$ (6)	AE $\times 10^2$ (7)
(0, 0, 1)	0.276****	1	0.2786	[0.273, 0.285]	0.94	0.068	0.260
		2	0.2749	[0.269, 0.281]	0.40	0.012	0.109
		3	0.2801	[0.274, 0.286]	1.46	0.162	0.403
		4	0.2794	[0.273, 0.286]	1.21	0.112	0.334
		Mean	0.2783	[0.272, 0.284]	0.80	0.049	0.222
(0, 1, 0)	-0.188****	1	-0.1854	[-0.190, -0.181]	1.30	0.060	0.245
		2	-0.1815	[-0.186, -0.177]	3.38	0.402	0.634
		3	-0.1796	[-0.184, -0.175]	4.38	0.675	0.822
		4	-0.1869	[-0.191, -0.182]	0.50	0.009	0.095
		Mean	-0.1833	[-0.188, -0.179]	2.39	0.202	0.449
(1, 0, 0)	-0.190****	1	-0.1949	[-0.199, -0.191]	2.37	0.204	0.452
		2	-0.1963	[-0.201, -0.192]	3.13	0.354	0.595
		3	-0.1929	[-0.197, -0.189]	1.32	0.063	0.252
		4	-0.1852	[-0.190, -0.181]	2.71	0.266	0.516
		Mean	-0.1923	[-0.197, -0.188]	1.03	0.038	0.196
(1, 1, 1)	0.079****	1	0.0888	[0.081, 0.096]	11.78	0.875	0.936
		2	0.0840	[0.076, 0.092]	5.81	0.213	0.462
		3	0.0958	[0.088, 0.104]	20.65	2.691	1.640
		4	0.0909	[0.083, 0.099]	14.44	1.315	1.147
		Mean	0.0899	[0.082, 0.098]	13.17	1.094	1.046
(1, 1, 0)	-0.311****	1	-0.3119	[-0.318, -0.306]	0.17	0.003	0.053
		2	-0.3106	[-0.317, -0.305]	0.23	0.005	0.072
		3	-0.3062	[-0.312, -0.300]	1.66	0.267	0.516
		4	-0.3037	[-0.310, -0.298]	2.45	0.581	0.762
		Mean	-0.3081	[-0.314, -0.302]	1.04	0.105	0.324
(1, 0, 1)	0.177****	1	0.1852	[0.177, 0.193]	4.82	0.725	0.852
		2	0.1788	[0.171, 0.186]	1.19	0.044	0.210
		3	0.1795	[0.172, 0.187]	1.60	0.080	0.282
		4	0.1855	[0.178, 0.193]	4.99	0.778	0.882
		Mean	0.1822	[0.175, 0.190]	3.15	0.310	0.557
(0, 1, 1)	0.202****	1	0.2049	[0.197, 0.212]	1.39	0.079	0.280
		2	0.1984	[0.191, 0.206]	1.84	0.138	0.372
		3	0.2061	[0.199, 0.214]	1.95	0.155	0.393
		4	0.2007	[0.193, 0.208]	0.71	0.021	0.143
		Mean	0.2025	[0.195, 0.210]	0.20	0.002	0.040

Fold estimates differ from each other by about as much as their standard errors: the standard deviation across the four fold estimates reaches 0.005 against fold standard errors of about 0.003, because each fold’s network is a different random fit. Averaging over folds is what makes the final estimate usable, and the paper’s convention of averaging the fold confidence bounds, rather than combining them as independent estimates, is a conservative acknowledgement of that.

5.3 Figure 5: DeDL against SDL

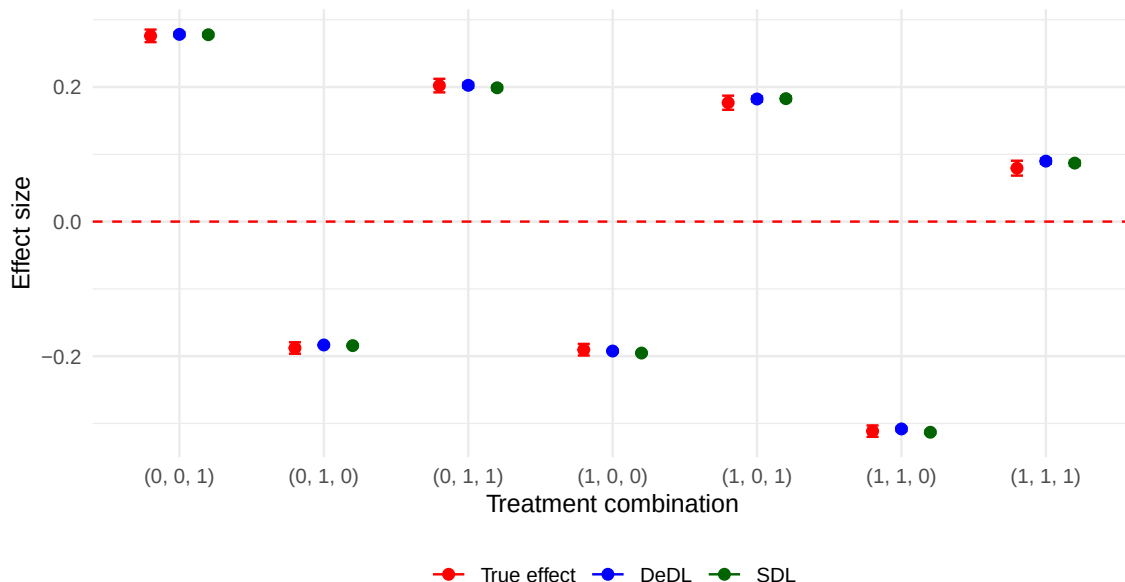


Figure 3: Detailed comparison between the SDL and DeDL estimators (paper’s Figure 5 layout): ground-truth effect with its confidence interval, DeDL, and SDL, for the seven non-baseline combinations. Bars are 95 percent intervals.

On the synthetic data the two estimators are close and both track the truth; the DeDL intervals are wider than the SDL intervals because the correction term adds variance, which is the price of removing the network’s bias. (The ground-truth bars use the standard error of the cell-mean difference; the notebook scales the cell standard deviation by a normal quantile of the p -value, which is not an interval.) The paper’s platform data show the SDL intervals to be too narrow; here, with a nearly noiseless synthetic outcome, the plug-in is already accurate.

6 Benchmarks and metrics

6.1 LA, LR, and PDL

Linear addition uses the cell means of the full data. Linear regression is fitted on the training rows of each fold (observed combinations only) and evaluated on the held-out fold; the pure network is trained the same way by the script with a 3-layer, 32-node ReLU architecture on (x, t) . Both report the mean contrast of predicted outcomes, with the authors’ inference: the standard deviation of the row-wise contrast over $\sqrt{n}$ and a Welch test between the two prediction vectors.

```
la_tab <- la_estimator(dat$y, key)
lr_folds <- lapply(sort(unique(dat$kfold)), function(f) {
  train <- dat[dat$kfold != f & key %in% observed_cells, ]
  lr_estimator(train, dat[dat$kfold == f, ], "y", tvar, xvar)
})
average_folds <- function(tabs) {
  bind_rows(tabs, .id = "fold") %>%
  group_by(combo) %>%
  summarise(estimate = mean(estimate), conf.low = mean(conf.low), conf.high = mean(conf.high),
    std.error = mean(std.error), p.value = max(p.value), .groups = "drop") %>%
  mutate(combo = factor(combo, levels = paper_order)) %>% arrange(combo) %>% mutate(combo = as.character(combo))
}
lr_tab <- average_folds(lapply(lr_folds, `[`, "table"))
pdl_paper <- py_run(
  output = file.path(derived, "pdl_paper.csv"),
```



```

args = c("--model pure --y y --t is_DP,is_LP,is_FYP --x-all-others --fold kfold",
        "--train-cells 000,001,010,100,111 --hidden 32 --depth 3 --epochs 50 --batch 128 --lr 0.001 --seed 1")
)
pdl_folds <- lapply(sort(unique(pdl_paper$fold)), function(f) pdl_estimator(pdl_paper[pdl_paper$fold == f, ]))
pdl_tab <- average_folds(lapply(pdl_folds, `[`, "table"))
dedl_tab <- pooled_dedl %>% transmute(combo = paper_order[-1], estimate, std.error, conf.low = ci_low_avg, conf.high = ci_high_avg)
  bind_rows(data.frame(combo = "000", estimate = 0, std.error = 0, conf.low = 0, conf.high = 0), .)
sdl_tab2 <- pooled_sdl %>% transmute(combo = paper_order[-1], estimate, std.error, conf.low, conf.high) %>%
  bind_rows(data.frame(combo = "000", estimate = 0, std.error = 0, conf.low = 0, conf.high = 0), .)
estimators <- list(LA = la_tab, LR = lr_tab, PDL = pdl_tab, SDL = sdl_tab2, DeDL = dedl_tab)
tab3 <- metrics_table(estimators, truth)

```

6.2 Table 3: comparison of the estimators of the ATE

Table 6: Comparison of different estimators of the ATE (paper’s Table 3 layout) on the synthetic data, with the MAPE printed by the authors’ notebook next to ours. MSE and MAE are scaled by constants, as in the paper.

Estimator	Unobserved treatment combinations					All treatment combinations				
	CDRu (1)	MAPEu, % (2)	MSEu $\times 10^4$ (3)	MAEu $\times 10^2$ (4)	Notebook MAPEu, %	CDR (5)	MAPE, % (6)	MSE $\times 10^4$ (7)	MAE $\times 10^2$ (8)	Notebook MAPE, %
LA	3/3	43.11	85.720	9.057	166.67	7/7	18.47	36.737	3.881	62.84
LR	3/3	20.85	27.114	4.810	21.99	7/7	27.74	33.234	5.410	24.89
PDL	3/3	48.51	108.321	10.299	3.87	7/7	25.51	47.249	4.911	5.98
SDL	3/3	1.86	0.168	0.367	1.95	7/7	2.88	0.210	0.410	2.79
DeDL	3/3	1.46	0.139	0.307	6.16	7/7	3.11	0.257	0.405	9.04

On the unobserved combinations the ranking is DeDL, SDL, LR, LA, PDL. Linear addition misses by 43.1 percent because the synthetic interactions are strong; linear regression by 20.8 percent because it cannot bend; the pure network by 48.5 percent because it never saw those combinations and must extrapolate without structure, which is the paper’s argument for a link function. The two structured estimators are within two percent, DeDL at 1.5 and SDL at 1.9; they are close because the plug-in is already nearly unbiased on this nearly noiseless outcome. Three of the notebook’s columns are not comparable: its linear addition mis-indexes the single effects (it adds the effects of (0, 0, 1) and (0, 1, 0) for the combination (1, 1, 0), and so on), its pure network is trained on all eight cells, and its DeDL departs from Algorithm 1 (Section 8.2).

6.3 Table 4: best-treatment identification

Each estimator names its best combination as the largest estimated ATE and reports the gap $\tau(t)$ to every other combination. DeDL and SDL use the difference of their influence functions (the package’s `contrasts`), LA adds variances, LR and PDL difference their predicted outcomes with a Welch test. The ground truth is the gap in cell means, tested cell against cell.

```

truth_tau <- {
  yb <- dat$y[key == best_true]
  do.call(rbind, lapply(paper_order, function(cc) {
    yt <- dat$y[key == cc]
    data.frame(combo = cc, diff = mean(yb) - mean(yt), p.value = if (cc == best_true) 1 else t.test(yb, yt)$p.value)
  }))
}
contrast_from_predictions <- function(pred_list, tab) {
  best <- tab$combo[which.max(tab$estimate)]
  bind_rows(lapply(pred_list, function(P) {
    do.call(rbind, lapply(paper_order, function(cc) {
      d <- P[, best] - P[, cc]
      data.frame(combo = cc, estimate = mean(d), std.error = sd(d) / sqrt(length(d)),
        p.value = if (cc == best) 1 else t.test(P[, best], P[, cc])$p.value)
    }))
  }), .id = "fold") %>% group_by(combo) %>%
  summarise(estimate = mean(estimate), std.error = mean(std.error), p.value = max(p.value), .groups = "drop") %>%
  mutate(conf.low = estimate - 1.96 * std.error, conf.high = estimate + 1.96 * std.error)
}
tau_lr <- contrast_from_predictions(lapply(lr_folds, `[`, "predictions"), lr_tab)

```

```

tau_pdl <- contrast_from_predictions(lapply(pdl_folds, `[`, "predictions"), pdl_tab)
tau_la <- {
  b <- which.max(la_tab$estimate)
  la_tab %>% transmute(combo, estimate = la_tab$estimate[b] - estimate,
    std.error = sqrt(la_tab$std.error[b]^2 + std.error^2),
    conf.low = estimate - 1.96 * std.error, conf.high = estimate + 1.96 * std.error)
}
tau_dedl <- fit_dedl$contrasts %>% filter(fold == "pooled") %>%
  transmute(combo = paper_order[-1], estimate, std.error, conf.low, conf.high) %>%
  bind_rows(data.frame(combo = "000", estimate = pooled_dedl$estimate[which.max(pooled_dedl$estimate)],
    std.error = pooled_dedl$std.error[which.max(pooled_dedl$estimate)]) %>%
    mutate(conf.low = estimate - 1.96 * std.error, conf.high = estimate + 1.96 * std.error), .)
fit_sdl_contrast <- {
  kb <- which.max(pooled_sdl$estimate)
  d <- fit_dedl$plugin[, kb] - fit_dedl$plugin
  folds <- sort(unique(fit_dedl$fold_id))
  bind_rows(lapply(seq_along(fit_dedl$labels), function(k) {
    fe <- sapply(folds, function(f) mean(d[fit_dedl$fold_id == f, k]))
    fs <- sapply(folds, function(f) sd(d[fit_dedl$fold_id == f, k]) / sqrt(sum(fit_dedl$fold_id == f)))
    data.frame(combo = paper_order[-1][k], estimate = mean(fe), std.error = sqrt(sum(fs^2)) / length(folds))
  })) %>% mutate(conf.low = estimate - 1.96 * std.error, conf.high = estimate + 1.96 * std.error)
}
tau_sdl <- bind_rows(data.frame(combo = "000", estimate = pooled_sdl$estimate[which.max(pooled_sdl$estimate)],
  std.error = pooled_sdl$std.error[which.max(pooled_sdl$estimate)]) %>%
  mutate(conf.low = estimate - 1.96 * std.error, conf.high = estimate + 1.96 * std.error), fit_sdl_contrast)
order_tau <- function(t) t[match(paper_order, t$combo), ]
tab4 <- bind_rows(lapply(list(LA = tau_la, LR = tau_lr, PDL = tau_pdl, SDL = tau_sdl, DeDL = tau_dedl), function(t) {
  combo_metrics(order_tau(t), truth_tau)
}), .id = "estimator")
best_named <- c(LA = la_tab$combo[which.max(la_tab$estimate)], LR = lr_tab$combo[which.max(lr_tab$estimate)],
  PDL = pdl_tab$combo[which.max(pdl_tab$estimate)], SDL = paper_order[-1][which.max(pooled_sdl$estimate)],
  DeDL = paper_order[-1][which.max(pooled_dedl$estimate)])

```

Table 7: Comparison of different estimators of best treatment identification (paper’s Table 4 layout). The true best combination is (0, 0, 1). Metrics compare the estimated gap to the best combination with the ground-truth gap over all eight combinations.

Estimator	Identified best	CDR (1)	MAPE, % (2)	MSE $\times 10^4$ (3)	MAE $\times 10^2$ (4)	Notebook MAPE, %
LA	(0, 0, 1)	8/8	32.13	32.145	3.396	127.79
LR	(0, 0, 1)	8/8	16.46	11.652	2.320	19.75
PDL	(1, 0, 1)	6/8	37.07	48.425	4.998	3.16
SDL	(0, 0, 1)	8/8	2.13	0.174	0.359	1.87
DeDL	(0, 0, 1)	8/8	1.55	0.139	0.289	1.69

7 Conditional effects by demographic group

The paper splits users by residence (rural or not) and phone price tier. The synthetic data do not carry that cross, so the notebook, and we, use the four groups defined by `gender_M` and `feed_model_1`. The group CATE of DeDL and SDL is the group mean of the influence functions, the LR and PDL CATEs are group means of the predicted contrasts, and the LA CATE adds the group’s single-treatment cell effects. The ground truth is the group’s cell means. As in the notebook’s Table 5 code, MAPE here averages over all non-baseline combinations without dropping insignificant ones.

```

grp <- paste0("G", 1 + 2 * dat$gender_M + dat$feed_model_1)
grp_names <- c(G1 = "female, feed model 2", G2 = "female, feed model 1", G3 = "male, feed model 2", G4 = "male, feed model 1")
lr_pred_all <- do.call(rbind, lapply(seq_along(lr_folds), function(i) {
  data.frame(.row = dat$.row[dat$kfold == sort(unique(dat$kfold))[i]], lr_folds[[i]]$predictions)
})) %>% arrange(.row)
pdl_pred_all <- do.call(rbind, lapply(seq_along(pdl_folds), function(i) {
  data.frame(.row = pdl_paper$.row[pdl_paper$fold == sort(unique(pdl_paper$fold))[i]], pdl_folds[[i]]$predictions)
}))

```

```

})) %>% arrange(.row)
cate_group <- function(g) {
  rows <- grp == g
  tr <- ground_truth_table(dat$y[rows], key[rows])
  la <- la_estimator(dat$y[rows], key[rows])$estimate
  dedl <- c(0, colMeans(fit_dedl$psi[rows, ]))
  sdl <- c(0, colMeans(fit_dedl$plugin[rows, ]))
  lr <- colMeans(as.matrix(lr_pred_all[rows, paste0("X", paper_order)])) - lr_pred_all[rows, "X000"])
  pdl <- colMeans(as.matrix(pdl_pred_all[rows, paste0("X", paper_order)])) - pdl_pred_all[rows, "X000"])
  list(n = sum(rows), truth = tr$diff, est = list(LA = la, LR = lr, PDL = pdl, SDL = sdl, DeDL = dedl))
}
cate_res <- lapply(sort(unique(grp)), cate_group)
names(cate_res) <- sort(unique(grp))
cate_metrics <- bind_rows(lapply(names(cate_res), function(g) {
  r <- cate_res[[g]]
  bind_rows(lapply(names(r$est), function(e) {
    err <- r$est[[e]] - r$truth
    idx_all <- 2:8; idx_u <- 6:8
    data.frame(group = g, n = r$n, estimator = e,
               mape_u = 100 * mean(abs(err[idx_u]) / r$truth[idx_u]), mse_u = mean(err[idx_u]^2), mae_u = mean(abs(err[idx_u])),
               mape = 100 * mean(abs(err[idx_all]) / r$truth[idx_all]), mse = mean(err[idx_all]^2), mae = mean(abs(err[idx_all])))
  })))
}))
tab5 <- cate_metrics %>% group_by(estimator) %>%
  summarise(across(c(mape_u, mse_u, mae_u, mape, mse, mae), ~ weighted.mean(.x, n)), .groups = "drop") %>%
  mutate(estimator = factor(estimator, levels = c("LA", "LR", "PDL", "SDL", "DeDL"))) %>% arrange(estimator)

```

Table 8: Comparison of different estimators of the CATE (paper’s Table 5 layout): group-size-weighted averages over the four demographic groups.

Estimator	Unobserved treatment combinations			All treatment combinations		
	MAPEu, % (1)	MSEu $\times 10^4$ (2)	MAEu $\times 10^2$ (3)	MAPE, % (4)	MSE $\times 10^4$ (5)	MAE $\times 10^2$ (6)
LA	322.83	87.947	8.989	138.35	37.691	3.853
LR	892.69	172.513	11.448	436.19	158.822	10.770
PDL	367.00	113.820	10.229	166.46	49.830	5.025
SDL	37.99	0.474	0.584	20.90	0.494	0.566
DeDL	39.98	0.455	0.610	22.36	0.515	0.606

Table 9: MAPE comparison of different estimators of the CATE for four groups (paper’s Table 6 layout). Groups: G1 = female, feed model 2; G2 = female, feed model 1; G3 = male, feed model 2; G4 = male, feed model 1. Group sizes in parentheses.

	Estimator	G1, % (12,389)	G2, % (12,608)	G3, % (12,516)	G4, % (12,487)
LA	Unobserved treatment combinations	52.05	1,163.05	32.04	34.57
LR	Unobserved treatment combinations	69.29	3,421.21	26.01	25.28
PDL	Unobserved treatment combinations	83.19	1,313.34	30.50	30.35
SDL	Unobserved treatment combinations	3.45	143.69	2.68	0.91
DeDL	Unobserved treatment combinations	4.00	151.72	2.35	0.59
LA	All treatment combinations	22.31	498.45	13.73	14.82
LR	All treatment combinations	145.59	1,527.86	29.59	29.81
PDL	All treatment combinations	63.27	566.76	16.38	15.10
SDL	All treatment combinations	16.01	63.38	2.50	1.31
DeDL	All treatment combinations	18.22	66.65	2.83	1.32

Within groups the structured estimators keep their advantage over LA and LR; SDL ranks first on the size-weighted MAPE. Group G2 has near-zero true effects for some combinations, so every estimator’s percentage error there is large; the notebook’s Table 6 shows the same group with errors above 100 percent. The notebook’s Table 5 code reports only the last group’s metrics (its loop overwrites the earlier groups), so its printed SDL and DeDL values, 0.88 and 0.76 percent for the unobserved combinations, correspond to our G4 column.

8 Implementation lessons

8.1 Figure 7: the training error as the compass

The paper’s Figure 7 retrains the fold-1 network at learning rate 10^{-4} and evaluates DeDL and SDL after every epoch. The script writes $\hat{\theta}$ for the held-out fold after each epoch; `est_dml_structured()` is then called fifty times.

```
split_file <- file.path(derived, "synthetic_fold1_split.csv")
if (!file.exists(split_file)) {
  write.csv(dat %>% select(-row) %>% mutate(split = ifelse(kfold == 1, "pred", "train")), split_file, row.names = FALSE)
}
theta_final <- py_run(
  input = split_file, output = file.path(derived, "theta_epochs_final.csv"),
  history = file.path(derived, "history_epochs.csv"), epoch_theta = file.path(derived, "theta_epochs.csv.gz"),
  args = c("--model structured --y y --t is_DP,is_LP,is_FYP --x-all-others --train-col split --train-value train",
    "--train-cells 000,001,010,100,111 --arch two_branch --hidden 20 --depth 3",
    "--epochs 50 --batch 128 --lr 0.0001 --val-split 0.1 --seed 10234")
)
theta_epochs <- read.csv(gzfile(file.path(derived, "theta_epochs.csv.gz")))
# one-split runs score every row; keep the held-out fold only
theta_epochs <- theta_epochs[theta_epochs$row %in% which(dat$kfold == 1), ]
history_epochs <- read.csv(file.path(derived, "history_epochs.csv"))
rows1 <- which(dat$kfold == 1)
lr_mape_fold1 <- combo_metrics(lr_folds[[1]]$table[-1, ], truth[-1, ])$mape
epoch_curve <- bind_rows(lapply(sort(unique(theta_epochs$epoch)), function(e) {
  te <- theta_epochs[theta_epochs$epoch == e, ]
  stopifnot(all(te$row == rows1))
  f <- est_dml_structured(as.matrix(te[, theta_cols]), dat$y[rows1], dat[rows1, tvar], targets = paper_order[-1],
    t_dist = t_dist_paper, ridge = ridge_main, contrast_best = FALSE)
  p <- f$estimates[f$estimates$fold == "pooled", ]
  data.frame(epoch = e,
    mape_dedl = mean(100 * abs(p$estimate - truth$diff[-1]) / abs(truth$diff[-1])),
    mape_sdl = mean(100 * abs(p$plugin - truth$diff[-1]) / abs(truth$diff[-1])))
})) %>% left_join(history_epochs %>% select(epoch, val_mse), by = "epoch") %>% mutate(mape_lr = lr_mape_fold1)
```

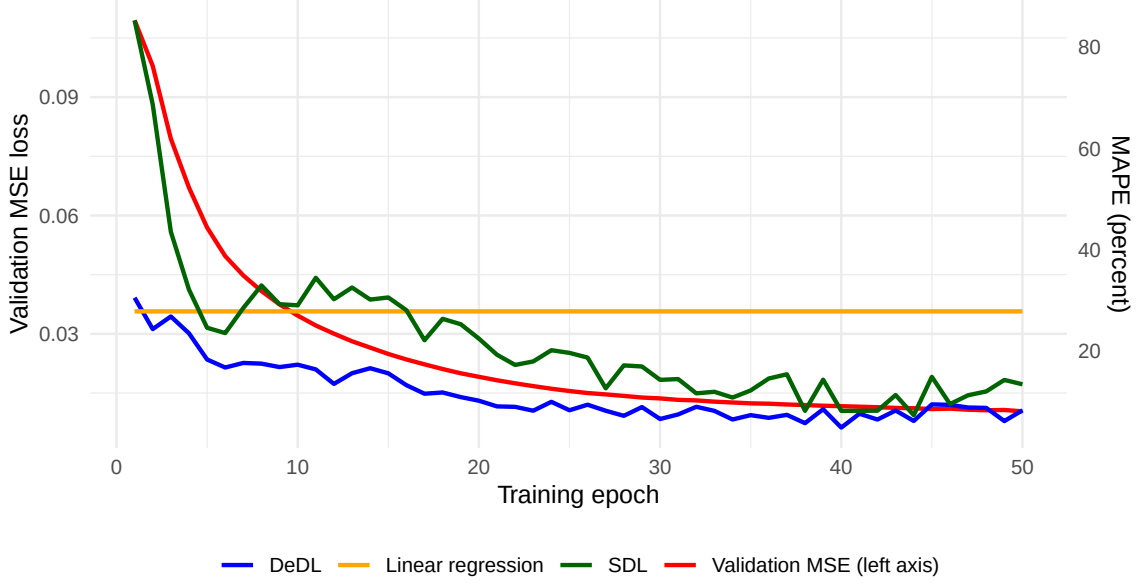


Figure 4: MAPE comparison with the training epoch (paper’s Figure 7 layout): validation MSE of the fold-1 network (left axis) and the MAPE over all combinations of DeDL, SDL, and linear regression (right axis) after each epoch.

At the slow learning rate the network is poorly trained for the first epochs and both structured estimators are worse than linear regression; DeDL falls below the LR line at epoch 2 and ends at 8.27 percent against 13.40 for SDL. The paper’s message is that the validation loss is the only observable guide to whether the first stage is good enough for debiasing to help, and that when the fit is poor, debiasing does not rescue it.

8.2 The authors’ notebook against the paper’s algorithm

The released notebook differs from Algorithm 1 in four places.

1. It trains on all rows of the three training folds, so the “unobserved” combinations are in the training data.
2. The shares in $\Lambda(x)$ are taken from the first five rows of a groupby over all eight cells, in sorted order, which are not the five observed combinations and sum to about 5/8.
3. The factor 2 in $\Lambda(x)$ is dropped and a ridge of 0.5 is added.
4. The DeDL row of Table 3 concatenates fold 4 four times.

The next table isolates these choices with the same `est_dml_structured()` call. The notebook’s Λ is reproduced by passing its shares halved as weights (the function’s factor 2 cancels the missing one) and its ridge.

```
theta_all <- py_run(
  output = file.path(derived, "theta_allcells.csv"), history = file.path(derived, "history_allcells.csv"),
  args = c("--model structured --y y --t is_DP,is_LP,is_FYP --x-all-others --fold kfold",
    "--arch two_branch --hidden 20 --depth 3 --epochs 50 --batch 128 --lr 0.001 --val-split 0.1 --seed 1")
)
theta_all_mat <- as.matrix(theta_all[, theta_cols])
shares_sorted <- prop.table(table(key))[c("000", "001", "010", "011", "100")]
t_dist_notebook <- data.frame(combo_matrix(c("000", "001", "100", "010", "111")), prob = as.numeric(shares_sorted) / 2)
names(t_dist_notebook)[1:3] <- tvar
variant_mape <- function(theta, t_dist, ridge) {
  f <- suppressWarnings(est_dml_structured(theta, dat$y, dat[, tvar], targets = paper_order[-1], t_dist = t_dist,
    fold_id = dat$kfold, ridge = ridge, contrast_best = FALSE))
  p <- f$estimates[f$estimates$fold == "pooled", ]
  ape <- 100 * abs(p$estimate - truth$diff[-1]) / abs(truth$diff[-1]); aps <- 100 * abs(p$plugin - truth$diff[-1]) / abs(truth$diff[-1])
  c(dedl_all = mean(ape), dedl_u = mean(ape[5:7]), sdl_all = mean(aps), sdl_u = mean(aps[5:7]))
}
```

```

}
variants <- bind_rows(
  data.frame(variant = "Paper: observed cells, uniform shares, factor 2, ridge 0.01", t(variant_map(theta_mat, t_dist_paper, ri
  data.frame(variant = "Paper design with the notebook's ridge 0.5", t(variant_map(theta_mat, t_dist_paper, 0.5))),
  data.frame(variant = "Notebook: all cells in training, uniform shares, factor 2, ridge 0.01", t(variant_map(theta_all_mat, t
  data.frame(variant = "Notebook: all cells, misaligned shares, no factor 2, ridge 0.5", t(variant_map(theta_all_mat, t_dist_no
)
pdl_all <- py_run(
  output = file.path(derived, "pdl_allcells.csv"),
  args = c("--model pure --y y --t is_DP,is_LP,is_FYP --x-all-others --fold kfold",
    "--hidden 32 --depth 3 --epochs 50 --batch 128 --lr 0.001 --seed 1")
)
pdl_all_tab <- average_folds(lapply(sort(unique(pdl_all$fold)), function(f) pdl_estimator(pdl_all[pdl_all$fold == f, ])$table))
pdl_all_metrics <- metrics_table(list(PDL = pdl_all_tab), truth)

```

Table 10: MAPE (percent) of DeDL and SDL under the paper’s algorithm and under the notebook’s departures from it.

Variant	MAPE, %			
	DeDL, all	DeDL, unobserved	SDL, all	SDL, unobserved
Paper: observed cells, uniform shares, factor 2, ridge 0.01	3.11	1.46	2.88	1.86
Paper design with the notebook’s ridge 0.5	3.00	1.84	2.88	1.86
Notebook: all cells in training, uniform shares, factor 2, ridge 0.01	3.48	2.02	2.95	1.39
Notebook: all cells, misaligned shares, no factor 2, ridge 0.5	3.38	1.86	2.95	1.39

Training on all cells improves the plug-in on the unobserved combinations (they are no longer unobserved), which is why the notebook’s SDL looks so good, and the heavy ridge with the misaligned shares shrinks the correction, which is why its DeDL cannot improve on SDL. The same holds for the pure network: trained on all cells it reaches a MAPE of 5.42 percent over all combinations and 2.80 on the “unobserved” ones (the notebook prints 5.98 and 3.87), against 25.5 and 48.5 when it must extrapolate. Under the paper’s own algorithm, with a small ridge, DeDL has MAPE 1.46 percent on the unobserved combinations against 1.86 for SDL.

9 Appendix D: synthetic experiments

The paper’s Appendix D validates DeDL on simulated combinatorial experiments. We port the base design: $m = 4$ experiments, $d_c = 10$ uniform covariates, outcome $c/(1 + \exp(-(\beta(x)\tilde{t}))) + \varepsilon$ with $\beta(x) = (x'C)^3$, $\tilde{t} = (1, t')'$, C uniform on $[-0.5, 0.5]$, c uniform on $[10, 20]$, uniform noise of ± 0.05 , six observed combinations (baseline, four singles, and $(1, 1, 0, 0)$) each with probability $1/6$, 2,000 training and 2,000 estimation users per replication. The network is the appendix’s single-branch one (one hidden layer of 10 ReLU units, outputs $\theta_0, \dots, \theta_4$, sigmoid, trained scale initialized at the maximum outcome), full-batch Adam at learning rate 0.05 for 2,000 epochs, restarted up to five times when the validation MSE exceeds 0.3. The estimators are LA, LR, SDL, and DeDL, scored on the estimation users against the true ATE function.

```

n_reps <- params$n_reps_appendix
designs <- lapply(seq_len(n_reps), function(r) appendix_d_design(seed = 100 + r))
appd_long <- bind_rows(lapply(seq_len(n_reps), function(r) {
  bind_rows(designs[[r]]$train %>% mutate(split = "train"), designs[[r]]$est %>% mutate(split = "est")) %>% mutate(rep = r)
}))
appd_in <- file.path(derived, "appd_in.csv")
if (!file.exists(appd_in) || params$rerun_python) write.csv(appd_long, appd_in, row.names = FALSE)
appd_theta <- py_run(
  input = appd_in, output = file.path(derived, "appd_theta.csv"), history = file.path(derived, "appd_history.csv"),
  args = c("--model structured --y y --t t1,t2,t3,t4 --x x1,x2,x3,x4,x5,x6,x7,x8,x9,x10 --group rep --train-col split --train-val

```

```

    "--arch single --hidden 10 --depth 1 --epochs 2000 --batch 0 --lr 0.05 --val-split 0.3 --restarts 5 --val-threshold 0
)
appd_results <- bind_rows(lapply(seq_len(n_reps), function(r) {
  des <- designs[[r]]
  est_rows <- which(appd_long$rep == r & appd_long$split == "est")
  th <- appd_theta[match(est_rows, appd_theta$.row), ]
  theta <- as.matrix(th[, c("theta_0", paste0("theta_", 1:4), "theta_c")])
  tv <- paste0("t", 1:4)
  keys <- des$truth_keys
  obs_keys <- combo_key(des$obs_t)
  truth_r <- data.frame(combo = keys, diff = des$truth, p.value = ifelse(keys == "0000", 1, 0))
  tdist <- data.frame(des$obs_t, prob = 1 / nrow(des$obs_t))
  fit <- est_dml_structured(theta, des$est$y, des$est[, tv], targets = des$all_t[-1, ], t_dist = tdist, ridge = ridge_main, control = control)
  p <- fit$estimates[fit$estimates$fold == "pooled", ]
  crit <- qnorm(0.975)
  dedl <- data.frame(combo = keys, estimate = c(0, p$estimate), conf.low = c(0, p$conf.low), conf.high = c(0, p$conf.high))
  sdl_se <- apply(fit$plugin, 2, sd) / sqrt(nrow(fit$plugin))
  sdl <- data.frame(combo = keys, estimate = c(0, p$plugin), conf.low = c(0, p$plugin - crit * sdl_se), conf.high = c(0, p$plugin + crit * sdl_se))
  # linear addition from the estimation sample's observed cells
  ekey <- combo_key(des$est[, tv])
  cell <- function(k) { v <- des$est$y[ekey == k]; c(mean = mean(v), var = var(v) / length(v)) }
  b <- cell("0000")
  la <- do.call(rbind, lapply(keys, function(k) {
    bits <- as.numeric(strsplit(k, "")[[1]])
    if (k %in% obs_keys && k != "0000") { s <- cell(k); est <- s["mean"] - b["mean"]; vr <- s["var"] + b["var"] }
    else if (k == "0000") { est <- 0; vr <- 0 }
    else { singles <- c("1000", "0100", "0010", "0001")[bits == 1]; est <- sum(sapply(singles, function(s) cell(s)["mean"] - b["mean"]))
    data.frame(combo = k, estimate = unname(est), conf.low = unname(est) - crit * sqrt(unname(vr)), conf.high = unname(est) + crit * sqrt(unname(vr)))
  }))
  lr <- lr_estimator(des$train, des$est, "y", tv, paste0("x", 1:10), combos = keys)$table
  ests <- list(LA = la, LR = lr, SDL = sdl, DeDL = dedl)
  unobs <- setdiff(keys, obs_keys)
  bind_rows(lapply(names(ests), function(e) {
    tab <- ests[[e]][match(keys, ests[[e]]$combo), ]
    mu <- combo_metrics(tab[keys %in% unobs, ], truth_r[keys %in% unobs, ])
    ma <- combo_metrics(tab[keys != "0000", ], truth_r[keys != "0000", ])
    data.frame(rep = r, estimator = e, cdr_u = mu$cdr_share, mape_u = mu$mape, mse_u = mu$mse, mae_u = mu$mae,
               cdr = ma$cdr_share, mape = ma$mape, mse = ma$mse, mae = ma$mae)
  })))
appd_tab <- appd_results %>% group_by(estimator) %>%
  summarise(across(c(cdr_u, mape_u, mse_u, mae_u, cdr, mape, mse, mae), mean), .groups = "drop") %>%
  mutate(estimator = factor(estimator, levels = c("LA", "LR", "SDL", "DeDL"))) %>% arrange(estimator)
appd_hist <- read.csv(file.path(derived, "appd_history.csv"))

```

Table 11: Synthetic experiments of the paper’s Appendix D (base design, $m = 4$, 20 replications): mean metrics over replications for the 10 unobserved and all 15 non-baseline combinations. Effects are in outcome units (scale c between 10 and 20).

Estimator	Unobserved treatment combinations				All treatment combinations			
	CDRu, %	MAPEu, %	MSEu	MAEu	CDR, %	MAPE, %	MSE	MAE
LA	76.5	63.83	0.034	0.137	76.3	54.63	0.025	0.112
LR	97.0	55.82	0.016	0.086	97.7	45.87	0.012	0.072
SDL	96.5	34.10	0.010	0.070	97.0	30.15	0.008	0.057
DeDL	93.5	20.30	0.009	0.052	93.7	17.91	0.006	0.040

With interactions built into the data through the sigmoid, linear addition and linear regression miss the unobserved combinations by a wide margin, while the structured estimators do far better, and DeDL improves on SDL by about a third in MAPE at the cost of a slightly lower correct-direction ratio (its intervals are wider). DeDL has a lower MAPE than SDL on the unobserved combinations in 80 percent of the replications; the validation MSE of the networks ends between 0.016 and 0.299, and 3 replications needed a restart. The

paper reports, with 200 replications and a longer tuning loop, MAPE below one percent for DeDL; our smaller run reproduces the ordering and the size of the gap to the linear benchmarks.

10 What matched, what did not, and what we learned

Table 12: Match between the authors' printed outputs on the synthetic data and this note.

Quantity	Authors' notebook	This note	Status
Table 2: relative effects (001, 010, 100, 111, 110, 101)	0.519, -0.353, -0.358, 0.149, -0.585, 0.332	0.519, -0.353, -0.358, 0.149, -0.585, 0.332	exact
Table 3: LA MAPE all / unobserved	62.84 / 166.67	18.47 / 43.11	notebook mis-indexes the single effects
Table 3: LR MAPE all / unobserved	24.89 / 21.99	27.74 / 20.85	close (fold averaging)
Table 3: PDL MAPE all / unobserved	5.98 / 3.87	5.42 / 2.80	same magnitude once trained on all cells, as the notebook does
Table 3: SDL MAPE all / unobserved	2.79 / 1.95	2.88 / 1.86	same magnitude
Table 3: DeDL MAPE all / unobserved	9.04 / 6.16	3.11 / 1.46	ours lower; notebook departs from Algorithm 1
Table 4: DeDL MAPE	1.69	1.55	same magnitude
Table 5, unobserved: SDL / DeDL MAPE	0.88 / 0.76 (last group only)	0.91 / 0.59 (G4)	same magnitude

Three lessons for the workflow of double machine learning with a structured first stage.

- **The influence function is only as good as $\Lambda(x)^{-1}$.** The orthogonal correction divides by the curvature of the loss in the nuisance direction. Where that curvature vanishes (saturated users), the correction is noise with infinite variance. A ridge is a bias-variance choice that the theory does not make for you; report the sensitivity.
- **The first-stage fit is observable, the nuisance error is not.** Figure 7 is the practical test: watch the validation loss and do not debias a model that has not converged.
- **Read the code, not only the paper.** The released notebook's departures (training set, assignment shares, ridge, a copy-paste in the results table, a mis-indexed linear-addition benchmark) change the ranking of the estimators. The package function makes each choice an explicit argument, which is what a replication should produce.

11 Appendix: formulas used

11.1 Gradients of the generalized sigmoid

For $u = \theta_0 + \sum_k \theta_k t_k$, $s = 1/(1 + e^{-u})$, $G = cs$: $\partial G / \partial \theta_0 = cs(1 - s)$, $\partial G / \partial \theta_k = cs(1 - s) t_k$, $\partial G / \partial c = s$. The loss gradient is $\ell_\theta = 2 G_\theta(\theta, \check{t}) (G(\theta, \check{t}) - y)$, and $\Lambda(x) = 2 \sum_{t \in \mathcal{T}^o} \nu(t) G_\theta(\theta(x), t) G_\theta(\theta(x), t)'$. The package's `link = "linear"` and `"logit"` replace G by u and by $s(u)$ with the corresponding gradients; `link = "custom"` takes both functions from the user.

11.2 The authors' inference conventions

- Ground truth and best treatment: cell-mean differences; pooled t -test if Levene's test does not reject at 5 percent, Welch otherwise.
- LA: the variance of a single-treatment effect is the sum of the two cell variances of the mean; combinations add the single effects and their variances.

- LR and PDL: the contrast of predicted outcomes has standard error $\text{sd}(\hat{y}_t - \hat{y}_0)/\sqrt{n}$ and its p -value is a Welch test between the two prediction vectors.
- DeDL and SDL: fold means and standard errors of ψ (or of H); the reported interval averages the fold bounds.
- Metrics: a combination with an insignificant ground truth contributes zero to MAPE, MSE, and MAE and counts as correctly identified when the estimate’s interval covers zero; otherwise the estimate must be significant with the right sign.

11.3 References

- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. 2018. “Double/Debiased Machine Learning for Treatment and Structural Parameters.” *The Econometrics Journal*.
- Farrell, Max H., Tengyuan Liang, and Sanjog Misra. 2020. “Deep Learning for Individual Heterogeneity: An Automatic Inference Framework.” arXiv:2010.14694.
- Farrell, Max H., Tengyuan Liang, and Sanjog Misra. 2021. “Deep Neural Networks for Estimation and Inference.” *Econometrica*.
- Ye, Zikun, Zhiqi Zhang, Dennis J. Zhang, Heng Zhang, and Renyu Zhang. 2025. “Deep Learning-Based Causal Inference for Large-Scale Combinatorial Experiments: Theory and Empirical Evidence.” *Management Science*.